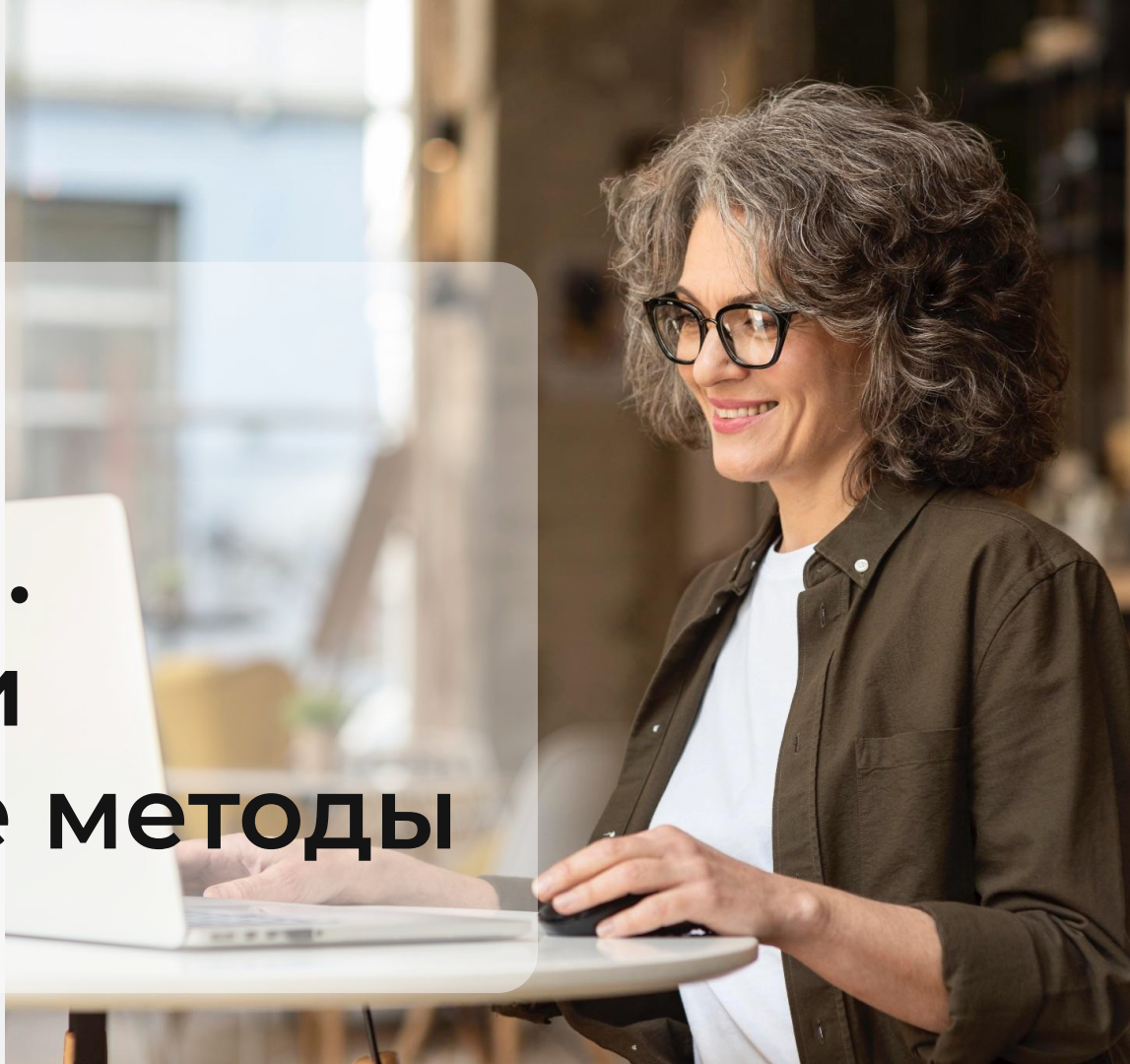


Python

Декораторы. Классовые и статические методы



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Принципы ООП
-  Наследование
-  Полиморфизм
-  Функция super
-  Функции isinstance и isinstance
-  Наследование от object

План занятия

- Декораторы
- Синтаксис `@decorator`
- Где применяются декораторы
- Поля класса
- Классовые методы
- Статические методы



ОСНОВНОЙ БЛОК





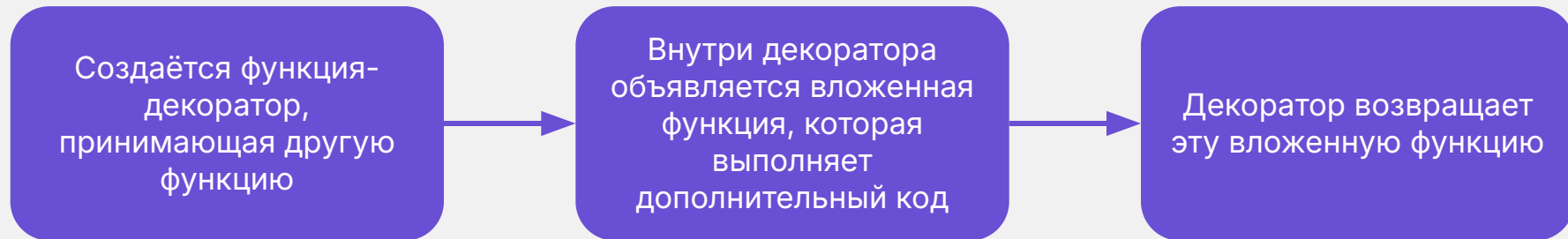
Декораторы



Декораторы

Это способ изменить поведение функции, не изменяя её код. Декоратор принимает функцию, добавляет к ней новую логику и возвращает изменённую версию

Работа декоратора



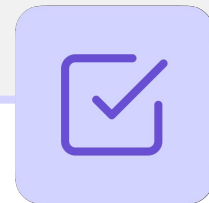
Пример

```
def simple_decorator(func):    # Функция-декоратор, принимает другую функцию
    def wrapper():            # Вложенная функция-обертка, добавляющая дополнительное поведение
        print("Перед вызовом функции")
        func()                # Вызываем переданную функцию
        print("После вызова функции")
    return wrapper            # Возвращаем изменённую функцию

def say_hello():
    print("Привет!")

decorated = simple_decorator(say_hello)    # Вызываем декоратор, теперь decorated = wrapper
print(decorated)
decorated()    # Теперь вызов say_hello() происходит через wrapper
```

wrapper



После применения декоратора результат (wrapper) можно сохранить в переменной с тем же именем, что и декорируемая функция. Тогда вместо вызова оригинальной функции будет вызываться функция wrapper, добавляющая дополнительный функционал.

Пример

```
def simple_decorator(func):    # Функция-декоратор, принимает другую функцию
    def wrapper():            # Вложенная функция-обертка, добавляющая дополнительное поведение
        print("Перед вызовом функции")
        func()                # Вызываем переданную функцию
        print("После вызова функции")
    return wrapper            # Возвращаем изменённую функцию

def say_hello():
    print("Привет!")

say_hello = simple_decorator(say_hello) # Вызываем декоратор, теперь decorated = wrapper
print(say_hello)
say_hello() # Теперь вызов say_hello() происходит через wrapper
```



ВОПРОСЫ





Синтаксис
@decorator

Синтаксис @decorator



Вместо явного вызова `decorated = simple_decorator(say_hello)` можно использовать `@` с именем декоратора перед определением функции

Пример

```
def simple_decorator(func):    # Функция-декоратор, принимает другую функцию
    def wrapper():            # Вложенная функция, добавляющая дополнительное поведение
        print("Перед вызовом функции")
        func()                # Вызываем переданную функцию
        print("После вызова функции")
    return wrapper            # Возвращаем изменённую функцию

@simple_decorator              # Эквивалентно say_hello = simple_decorator(say_hello)
def say_hello():
    print("Привет!")

say_hello()
```


Применение декоратора



Декораторы полезны, когда нужно добавлять дополнительное поведение к функциям. Один из распространённых примеров — декоратор для измерения времени выполнения.

Пример

```
import time

def timing_decorator(func):
    def wrapper():
        start_time = time.time() # Засекаем время перед выполнением функции
        func() # Вызываем декорируемую функцию
        end_time = time.time() # Засекаем время после выполнения
        print(f"Функция {func.__name__} выполнялась {end_time - start_time:.5f} секунд")
    return wrapper

@timing_decorator # Применение декоратора
def slow_function():
    time.sleep(2) # Имитация долгой операции
    print("Функция выполнена")

slow_function()
```



ВОПРОСЫ





Где применяются декораторы

Основные области применения декораторов



Логирование вызовов функций



Измерение времени выполнения



Ограничение частоты вызовов
(Throttling)



Проверка и валидация входных
данных



Автоматическое повторение
выполнения (Retry)



Ограничение доступа



Кеширование результатов



Автоматическое изменение данных



ВОПРОСЫ





ЗАДАНИЯ





Выберите верный вариант ответа

1. **Что делает декоратор?**
 - a. Изменяет код функции внутри её тела
 - b. Добавляет дополнительную логику к функции без изменения её тела
 - c. Копирует поведение функции в другую переменную

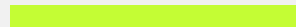


Выберите верный вариант ответа

1. Что делает декоратор?
 - a. Изменяет код функции внутри её тела
 - b. Добавляет дополнительную логику к функции без изменения её тела
 - c. Копирует поведение функции в другую переменную



Ответьте на вопрос



2. Что делает конструкция `@decorator_name`?



Ответьте на вопрос



2. Что делает конструкция `@decorator_name`?

Ответ: Заменяет функцию на результат работы декоратора



ВОПРОСЫ





Поля класса



Поле класса (атрибут класса)

Это переменная, которая принадлежит самому классу, а не отдельному объекту. Все экземпляры класса могут получить к ней доступ, и при этом значение хранится в одном месте — внутри класса.

Поле класса



Синтаксис

```
class ClassName:  
    class_attribute = value # поле  
класса
```

Пояснение

- `class_attribute` — поле класса, общее для всех объектов
- `value` — значение этого поля

Пример

```
class Book:
    library_name = "Central Library" # поле класса

    def __init__(self, title, author):
        self.title = title # поле объекта
        self.author = author # поле объекта
```


Использование полей класса



Для хранения настроек, категорий, счётчиков, общих значений



Когда значение одинаково для всех объектов



ВОПРОСЫ





Доступ к полям

Изменение полей класса



Поля класса можно читать и изменять как внутри класса, так и снаружи — через класс или объект. Но при этом поведение может отличаться — особенно при изменении значений.

1. Чтение поля класса



Напрямую через класс: `ClassName.attribute`



Через объект: `object_name.attribute`



Внутри методов класса — через `self.attribute` или `ClassName.attribute`

Пример чтения поля класса

```
class Book:
    library_name = "Central Library" # поле класса

    def __init__(self, title, author):
        self.title = title
        self.author = author

    def show_library(self):
        print(self.library_name) # доступ через self
        print(Book.library_name) # доступ через имя класса

print(Book.library_name) # доступ через класс
book = Book("1984", "George Orwell")
print(book.library_name) # доступ через объект
book.show_library()      # доступ изнутри
```

2. Изменение значения через класс

```
class Book:
    library_name = "Central Library" # поле класса

    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book("1984", "George Orwell")
book2 = Book("Brave New World", "Aldous Huxley")
Book.library_name = "City Library"
print(Book.library_name)
print(book.library_name)      # у объекта тоже изменилось
print(book2.library_name)     # у объекта тоже изменилось
```

3. Создание поля через объект



Если попытаться изменить значение поля через объект, на самом деле будет создано **новое поле в конкретном объекте**, не затронув поле класса.

Пример создания поля через объект

```
class Book:
    library_name = "Central Library" # поле класса

    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book("1984", "George Orwell")
book2 = Book("Brave New World", "Aldous Huxley")

book.library_name = "Private Shelf" # создаёт новое поле в book
print(book.library_name) # Private Shelf – новое значение в объекте
print(Book.library_name) # поле класса не изменилось
print(book2.library_name) # также прочитает поле класса
```



ВОПРОСЫ





ЗАДАНИЯ





Выберите верные варианты ответа

Укажите верные утверждения о полях класса:

- a. Поле класса общее для всех объектов
- b. Поле класса создаётся внутри метода `__init__()`
- c. Изменение поля через объект всегда меняет поле класса
- d. Поле класса можно прочитать через объект



Выберите верные варианты ответа

Укажите верные утверждения о полях класса:

- a. Поле класса общее для всех объектов
- b. Поле класса создаётся внутри метода `__init__()`
- c. Изменение поля через объект всегда меняет поле класса
- d. Поле класса можно прочитать через объект



ВОПРОСЫ





Классовые методы



Классовый метод

Это метод, который работает не с конкретным объектом, а с самим классом. Он получает доступ не к `self`, а к `cls` — ссылке на класс, из которого был вызван.

Важно



Чтобы объявить классový метод, используется декоратор `@classmethod`.

Применение классовых методов



Работа с полями класса (например, счётчиками или общими настройками)



Добавление поведения, относящегося к самому классу, а не к отдельному экземпляру

Классовые методы



Синтаксис

```
class ClassName:
    @classmethod
    def method_name(cls, ...):
        ...
```

Пояснение

- `@classmethod` — декоратор, помечающий метод как классový
- `cls` — ссылка на сам класс (аналог `self`, но для класса)

Пример: счётчик созданных объектов

```
class Book:
    total_books = 0 # поле класса

    def __init__(self, title, author):
        self.title = title
        self.author = author
        Book.total_books += 1

    @classmethod
    def show_total(cls):
        print(f"Total books: {cls.total_books}")
        # print(cls.title) # Нельзя обратиться к полю объекта

Book.show_total() # Вызов через класс
book1 = Book("1984", "George Orwell")
book2 = Book("Brave New World", "Aldous Huxley")

Book.show_total() # Вызов через класс
book1.show_total() # Вызов через объект
```

Пример: поведение, связанное с классом

```
class Book:
    default_format = "PDF"
    default_language = "en"

    @classmethod
    def show_defaults(cls):
        print(f"Format: {cls.default_format}, Language: {cls.default_language}")

    @classmethod
    def reset_defaults(cls):
        cls.default_format = "PDF"
        cls.default_language = "en"

Book.default_format = "EPUB" # Изменение формата
Book.show_defaults()         # Просмотр текущих настроек
Book.reset_defaults()        # Сброс до базовых настроек
Book.show_defaults()         # Просмотр текущих настроек
```



ВОПРОСЫ





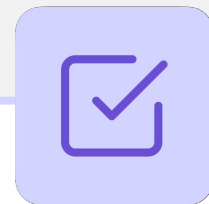
Статические методы



Статический метод

Это метод, который не зависит ни от объекта, ни от самого класса. Он работает как обычная функция, но помещён внутрь класса для логической связанности с ним.

Важно



Чтобы объявить статический метод, используется декоратор `@staticmethod`

Использование статических методов



Когда функция логически относится к классу, но не использует ни `self`, ни `cls`



Чтобы сгруппировать связанную логику внутри класса, а не выносить её в отдельные функции вне класса

Статические методы



Синтаксис

```
class ClassName:
    @staticmethod
    def method_name(...):
        ...
```

Пояснение

- `@staticmethod` — декоратор, превращающий метод в статический
- Такой метод **не получает ни self, ни cls**

Пример: вспомогательная функция внутри класса

```
class Book:
    def __init__(self, title):
        self.title = title

    @staticmethod
    def is_title_valid(title):
        return isinstance(title, str) and len(title) > 0

print(Book.is_title_valid("1984"))      # True
print(Book.is_title_valid(""))         # False
print(Book.is_title_valid(123))        # False
```

Такой метод можно вызывать и через класс (`Book.is_title_valid()`), и через объект — результат будет одинаков

Пример: логически связанные, но независимые операции

```
class MathHelper:
    @staticmethod
    def square(x):
        return x * x

    @staticmethod
    def cube(x):
        return x * x * x

print(MathHelper.square(5))
print(MathHelper.cube(3))
```

Сравнение типов методов

Тип метода	Обычный метод	Классовый метод	Статический метод
Декоратор	–	@classmethod	@staticmethod
Первый аргумент	self	cls	–
Доступ	К полям и методам объекта	К полям и методам класса	–
Когда использовать	Когда метод работает с конкретным экземпляром	Когда метод работает с классом или создаёт объекты по шаблону	Когда функция логически относится к классу, но ничего не использует



ВОПРОСЫ





ЗАДАНИЯ





Выберите верные варианты ответа

1. Укажи верные утверждения о `@classmethod`:
 - a. Метод `@classmethod` получает первым аргументом `self`
 - b. Метод `@classmethod` может создавать объект класса
 - c. Метод `@classmethod` имеет доступ к атрибутам конкретного объекта
 - d. Метод `@classmethod` вызывается только через класс, а не через объект
 - e. Метод `@classmethod` имеет доступ к классу



Выберите верные варианты ответа

1. Укажи верные утверждения о `@classmethod`:
 - a. Метод `@classmethod` получает первым аргументом `self`
 - b. Метод `@classmethod` может создавать объект класса
 - c. Метод `@classmethod` имеет доступ к атрибутам конкретного объекта
 - d. Метод `@classmethod` вызывается только через класс, а не через объект
 - e. Метод `@classmethod` имеет доступ к классу



Выполните задание



2. Найдите ошибку в коде:

```
class Person:
    @staticmethod
    def greet(self):
        print("Hello!")
```



Выполните задание

2. Найдите ошибку в коде:

```
class Person:
    @staticmethod
    def greet(self):
        print("Hello!")
```

Ответ: у статического метода не должно быть параметра `self`



ВОПРОСЫ

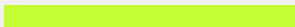




ПРАКТИЧЕСКАЯ РАБОТА



1. Расстояние между городами



Создайте класс `City`, представляющий город с координатами.

- У каждого города есть поля `name`, `latitude`, `longitude`.
- Добавьте строковое представление объекта.
- Добавьте метод `distance(city1, city2)`, который возвращает кортеж (`latitude`, `longitude`) между двумя городами.
- Проверьте расстояние между двумя городами.

Пример вывода:

City: Berlin (52.52, 13.4)

City: Paris (48.85, 2.35)

Distance: 14.72



ДОМАШНЕЕ ЗАДАНИЕ



Домашнее задание

1. Счётчик экземпляров

Создайте класс `User`, представляющий пользователя.

- При создании должны указываться логин (`username`) и пароль (`password`).
- У класса должно быть поле `total_users`, хранящее общее количество созданных пользователей.
- При каждом создании нового объекта `User`, счётчик должен увеличиваться.
- Добавьте метод `get_total()`, возвращающий количество пользователей.
- Проверьте, что счётчик работает.

Пример вывода:

```
Total users: 2
```

Домашнее задание

2. Проверка данных пользователя

Доработайте класс User.

- Добавьте валидации полей при создании.
- Имя должно быть непустой строкой.
- Пароль должен быть строкой длиной не менее 5 символов.
- Если данные некорректны — выбрасывайте ValueError.
- Добавьте строковое представление объекта.
- Проверьте работу класса с разными значениями.

Пример вызова:

```
user1 = User("alice", "secret")
user2 = User("bob", "qwe")
```

Пример вывода:

```
User: alice
...
ValueError: Invalid password: 'qwe'.
```

Заключение

